Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.2 / COMPANION WORKBOOK

Configure DNS and time

Service discovery, zones, records, domain time & troubleshooting

Learn how DNS lets every domain member find its domain controllers, then build dependable internal name resolution and time on DC01 and DC02: set each controller's DNS order, configure forward and reverse zones, manage service records, make the PDC emulator the domain's time source, and prove both in the air-gapped training network.

MATERIALS EDITION 1.4 / COURSE 1.2

Updated 9 October 2026 · sbd-dns-configuration

NAME _____ / DATE _____

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Map names to the network

02 Configure forward and reverse zones

03 Create and maintain service records

04 Prove resolution and directory discovery

05 Make SBD-DC01 the domain's time source

06 Find the fault and record the fix

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

The addresses and names below continue the training example. Match every change to your own address plan. This network has no internet DNS or time dependency: names resolve on the domain controllers, and time comes from the PDC emulator, SBD-DC01.

Prerequisites

- ☐ Complete course 1.1, the DC01 + DC02 identity build, with both domain controllers healthy. Both were promoted with the DNS server option selected, and SBD-DC02 still uses SBD-DC01 for DNS.
- ☐ Use an isolated Windows Server practice environment and an account delegated to manage DNS.
- ☐ Continue the existing sbd.test example: SBD-DC01 is 10.0.10.10 and SBD-DC02 is 10.0.10.11 on 10.0.10.0/24.
- ☐ The network's routing and VLAN segmentation are already in place, with a documented service-path register.

What this course builds

- Explain how DNS lets members find domain controllers, and plan internal DNS and client settings.
- Configure secure AD-integrated zones and maintain records.
- Test both resolvers, directory discovery, and client fallback.
- Make the PDC emulator the domain's time source and verify the time hierarchy end to end.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-DC01 — name, address, OS build	
SBD-DC02 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 25 MIN

Map names to the network

> **WHY THIS MATTERS** Domain Name System (DNS) turns names into addresses, and for Active Directory it does one more job: it tells every domain member where its domain controllers are. Start from the working directory build, learn what DNS answers and what it doesn't, set each controller's DNS client order, and record which server owns each name and how a client finds it.

In this lesson you will

- Explain how DNS lets a domain member find a domain controller, and what a DNS answer does not prove.
- Set each domain controller's DNS client order and point domain members only at internal DNS.
- Inventory the existing sbd.test and _msdcs zones without creating duplicates.
- Document the forwarding and recursion decision for an air-gapped network.

Follow the build

1. Start with what DNS answers. An A record maps a host name to an IPv4 address (AAAA maps to IPv6), so SBD-DC01.sbd.test resolves to 10.0.10.10. Service (SRV) records advertise which servers offer a service: a domain member asks for _ldap._tcp.dc._msdcs.sbd.test, resolves a listed controller's name, contacts it, and only then authenticates. DNS never checks a password, and it works on an isolated network with no internet access.
2. Record the domain, each subnet, both DNS addresses, and the owner of each service name. Keep host records and service aliases distinguishable.
3. On each domain controller, confirm the DNS Server service is installed and running; an address typed into a network adapter installs nothing. Then open DNS Manager and review the sbd.test and _msdcs.sbd.test zones that promotion created, without creating duplicates. The DNS delegation warning from course 1.1's promotion is expected: no parent zone exists above the isolated .test namespace, so no action is needed.
4. Set each controller's DNS client order. Promotion pointed SBD-DC01 at itself (the loopback address, 127.0.0.1) while it was the only controller, and SBD-DC02 pointed at SBD-DC01 so it could find the domain. Run the replication summary first; once replication succeeds in both directions, list the partner controller first and the loopback address second on each, and never an outside resolver. Domain members list both controllers; a public resolver is not an alternate for this environment.
5. Confirm the network's service-path register permits DNS: every networked system to both domain controllers on TCP and UDP 53 (rule F01 in Inside the Network). A member has to find a controller before any Kerberos, LDAP, or Group Policy traffic can start.
6. Decide whether any separate internal namespace needs a conditional forwarder. With no recursive dependencies, document a no-recursion design; do not add internet forwarders or delete every root hint as a shortcut.

Check your result

```
Invoke-Command SBD-DC01,SBD-DC02 { Get-WindowsFeature DNS; Get-Service DNS }
repadmin /replsummary
Get-DnsClientServerAddress -AddressFamily IPv4 -CimSession SBD-DC01,SBD-DC02
```

© 2026 Secure By Design Team LLC.

```
# After replication succeeds both ways (adapter name varies):
Set-DnsClientServerAddress -CimSession SBD-DC01 -InterfaceAlias Ethernet `
  -ServerAddresses 10.0.10.11,127.0.0.1
Set-DnsClientServerAddress -CimSession SBD-DC02 -InterfaceAlias Ethernet `
  -ServerAddresses 10.0.10.10,127.0.0.1
Get-DnsClientServerAddress -AddressFamily IPv4 -CimSession SBD-DC01,SBD-DC02
Get-DnsServerZone -ComputerName SBD-DC01
```

WHAT SUCCESS LOOKS LIKE Both controllers run the DNS Server service and, once replication succeeds both ways, list their partner first and the loopback address second. Members list both controllers only; the AD zones, the recursion decision, and the register rows that permit DNS are recorded.

Your record

Evidence to keep: Save the address/name worksheet, each controller's DNS client order and the reason for it, and the initial zone inventory before changing anything else.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Which DNS servers should a domain member use in this lab?

REMEMBER DNS finds domain controllers through SRV records; it never checks a password. Each controller lists its partner, then itself; members list both controllers only. The AD zones are inventoried, not recreated, and the delegation warning is expected. The forwarding and recursion decision is written down first.

Microsoft · Install and configure DNS Server

Microsoft · What is DNS?

Microsoft · Best practices for DNS client settings

Microsoft · Locating domain controllers

Microsoft · Service overview and network port requirements

Microsoft · AD DS configuration wizard page descriptions

Microsoft · Install-ADDSForest

© 2026 Secure By Design Team LLC.

LESSON 02 / ABOUT 20 MIN

Configure forward and reverse zones

WHY THIS MATTERS A zone holds the records for one part of the namespace. Forward lookup answers where a name lives. Reverse lookup answers which name is assigned to an address. AD integration lets the directory replicate zone data between the selected DNS servers.

In this lesson you will

- Confirm sbd.test is AD-integrated with the intended replication scope.
- Create a secure, AD-integrated reverse zone for 10.0.10.0/24.
- Explain how zone data moves between the two DNS hosts.

Follow the build

1. In DNS Manager, inspect sbd.test Properties. Confirm that it is AD-integrated and choose the domain or forest replication scope that matches the DNS servers intended to host it. Domain-wide zones live in the DomainDnsZones partition and forest-wide zones in ForestDnsZones, the partitions you saw in course 1.1's replication and DCDiag output.
2. Use secure dynamic updates for the AD-integrated zone. Review delegation and permissions before broadening who can change records.
3. Under Reverse Lookup Zones, create an IPv4 primary AD-integrated zone for network ID 10.0.10 if an equivalent zone does not already exist. Select the approved replication scope and secure updates.
4. Review zone-transfer settings. AD replication between these DCs does not require enabling transfers to arbitrary DNS servers; restrict any separately required secondary-server transfers explicitly.

Check your result

```
Get-DnsServerZone -ComputerName SBD-DC01 |
  Select-Object ZoneName, ZoneType, IsDsIntegrated, DynamicUpdate, ReplicationScope
Get-DnsServerZone -ComputerName SBD-DC02
```

WHAT SUCCESS LOOKS LIKE The intended forward and reverse zones appear on both DNS hosts after replication. The reverse zone for this /24 is 10.0.10.in-addr.arpa.

Your record

Evidence to keep: Capture the zone type, update policy, replication scope, and transfer policy for each zone.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	

RECORD	YOUR OBSERVATION
Evidence file names	
Open items or findings	

Think it through

How does an AD-integrated zone reach the other DNS-enabled domain controller in its replication scope?

REMEMBER Forward and reverse zones are AD-integrated with a deliberate replication scope. Dynamic updates are secure-only. Zone transfers stay restricted; directory replication moves the data.

Microsoft · Manage DNS zones

Microsoft · Active Directory-integrated DNS zones

LESSON 03 / ABOUT 20 MIN

Create and maintain service records

WHY THIS MATTERS Build a small, intentional record set. A host record identifies a machine; an alias provides a stable service name without making that alias a separate server.

In this lesson you will

- Distinguish host records (A and PTR) from service aliases (CNAME).
- Create a host record with its PTR and a service alias that match the plan.
- Maintain records with recorded ownership, lifetimes, and reasons.

Follow the build

1. Review the DC A and PTR records first. Correct mismatches through the owning host and DNS administration process rather than blindly adding another record with the same name.
2. Choose one approved service host from your address plan. In sbd.test, use New Host (A or AAAA), enter its actual name and address, and select the associated PTR option when the reverse zone is ready.
3. For a service alias, use New Alias (CNAME) and select the intended fully qualified host. For the later PKI course, reserve pki.sbd.test for the approved internal publication host; do not point it at the offline root.
4. Set the record lifetime to the operational plan, record static ownership, and remove only records confirmed obsolete. An alias cannot coexist with other record types at the same name.

Check your result

```
Get-DnsServerResourceRecord -ComputerName SBD-DC01 -ZoneName sbd.test
Resolve-DnsName SBD-DC01.sbd.test -Type A -Server 10.0.10.10
Resolve-DnsName 10.0.10.10 -Type PTR -Server 10.0.10.10
```

WHAT SUCCESS LOOKS LIKE Forward and reverse responses agree with the address plan. Each alias reaches the intended host; creating a DNS name alone does not configure its service.

Your record

Evidence to keep: Keep a record table with owner, type, target, time to live, and reason for the change.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	

RECORD	YOUR OBSERVATION
Open items or findings	

Think it through

What does creating pki.sbd.test as an alias accomplish?

> **REMEMBER** Host records identify machines; aliases provide stable service names. pki.sbd.test is reserved for the internal publication host, never the offline root. Every record carries an owner, a lifetime, and a reason.

Microsoft · Manage DNS resource records

LESSON 04 / ABOUT 30 MIN

Prove resolution and directory discovery

> **WHY THIS MATTERS** Test each server explicitly before testing client fallback. A successful cached answer does not prove that both DNS servers hold the right records, and a correct DNS answer proves neither a successful sign-in nor access to a protected resource.

In this lesson you will

- Query each DNS server explicitly instead of trusting cached answers.
- Verify the locator (SRV) records that let clients discover domain controllers.
- Run a controlled failover test and record the result.
- Optionally capture a locator query and compare it with the model's packet card.

Follow the build

1. From a domain client or either controller, query each DNS server by address for both controllers' host records, so each result tells you which server answered. Expect SBD-DC01.sbd.test at 10.0.10.10 and SBD-DC02.sbd.test at 10.0.10.11 from both servers.
2. Query the domain controller locator service (SRV) records against both servers. These records let a client discover domain controllers; an ordinary host lookup does not prove discovery works, and a locator answer does not by itself prove that every listed controller is reachable.
3. Run the DNS diagnostics against every domain controller, then the replication summary. Classify root-hint, forwarder, and delegation findings against the documented isolated design instead of treating them as faults, and investigate everything else.
4. During an approved lab test, make one DNS server unavailable to the test client and verify resolution through the other. Restore connectivity and repeat for the opposite server. Do not interrupt shared production services for this exercise.
5. Optional: see the locator query on the wire. On SBD-DC02, in an elevated PowerShell window, capture port 53 with Packet Monitor (pktmon), which is built into Windows Server, while you query SBD-DC01, so the request crosses the network. Expect a query from 10.0.10.11 to 10.0.10.10 on UDP port 53 and an answer naming both controllers on port 389. Read the text file, or open the .pcapng file in a packet analyzer such as Wireshark, and keep the capture with your evidence; it holds only the traffic the filter allowed.

Check your result

```
foreach ($server in '10.0.10.10','10.0.10.11') {
  Write-Output "DNS server: $server"
  Resolve-DnsName 'SBD-DC01.sbd.test' -Type A -Server $server
  Resolve-DnsName 'SBD-DC02.sbd.test' -Type A -Server $server
  Resolve-DnsName '_ldap._tcp.dc._msdcs.sbd.test' -Type SRV -Server $server
}
dcdiag /test:DNS /e /v
repadmin /replsummary
# Optional, on SBD-DC02: capture the locator query
New-Item -ItemType Directory C:\Evidence -Force
pktmon filter remove
pktmon filter add DNS -p 53
pktmon start --capture --comp nics --pkt-size 0 --file-name C:\Evidence\dc-locator.etl
```

```
Resolve-DnsName '_ldap._tcp.dc._msdcs.sbd.test' -Type SRV -Server 10.0.10.10
pktmon stop
pktmon etl2txt C:\Evidence\dc-locator.etl --out C:\Evidence\dc-locator.txt
pktmon etl2pcap C:\Evidence\dc-locator.etl --out C:\Evidence\dc-locator.pcapng
pktmon filter remove
```

> **WHAT SUCCESS LOOKS LIKE** Both DNS servers return both controllers' host records and the domain controller locator records, the DCDiag DNS findings are explained, replication is healthy, and the client still resolves internal names when either resolver is unavailable. If you ran the capture, it shows the query from 10.0.10.11 and the answer from 10.0.10.10.

Your record

Evidence to keep: Keep paired lookup results with the server queried, the DCDiag and replication output, the failover test record with start, restore, and final verification times, any capture file, and a short statement of what the results do and don't establish: DNS helps users find the identity service, but a correct answer proves nothing about authentication.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why query each DNS server directly?

> **REMEMBER** Both resolvers answer the host and SRV queries correctly. dcdiag and repadmin confirm DNS and replication health, with isolated-design findings explained. The failover test is documented with times, not just a passing screenshot.

Microsoft · Locating domain controllers

Microsoft · Resolve-DnsName

Microsoft · Dcdiag

Microsoft · Packet Monitor (pktmon)

LESSON 05 / ABOUT 30 MIN

Make SBD-DC01 the domain's time source

> **WHY THIS MATTERS** Kerberos tolerates minutes of skew, not more, and an air-gapped network has no internet time to lean on. Make the PDC emulator, SBD-DC01, the domain's authoritative time source and let the domain hierarchy carry its time to everything else.

In this lesson you will

- Configure SBD-DC01, the PDC emulator, as the domain's reliable time source.
- Keep every other system on the domain hierarchy, with nothing overriding it.
- Document how DC01's clock is set and checked, and the drift threshold the model calls for.

Follow the build

1. Confirm which domain controller holds the PDC emulator role — SBD-DC01 in this sample. The PDC emulator of the forest root domain sits at the top of the Windows Time hierarchy, so it is the one system you configure; everything else follows it.
2. On SBD-DC01, mark Windows Time as a reliable time source with no external peer, so it serves time from its own clock. In an air-gapped network that clock is the reference by declaration: record how it is set and checked — for example, compared on a schedule against an approved reference and corrected by an administrator, with each correction recorded — and the drift threshold you will hold it to.
3. Turn off the hypervisor's time synchronization for both domain controller virtual machines, so the host clock cannot silently override the PDC emulator. SBD-DC02 syncs from the PDC emulator, every other domain member keeps the default domain-hierarchy sync from the domain controllers, and the domain-joined Linux servers in later courses point chrony or ntpd at both domain controllers.
4. Confirm the network's service-path register permits time — every member to both domain controllers on UDP 123 — and write the time design into the build record: course 1.3's hardening Group Policy objects (GPOs) must leave it intact, and if a Security Technical Implementation Guide (STIG) value disagrees with it, that course's deviation record is where the decision lives.
5. Verify the chain end to end: SBD-DC01 serving as the reliable source, SBD-DC02 and a member server sourcing from a domain controller, and offsets within your recorded threshold.

Check your result

```
Get-ADDomain | Select-Object PDCEmulator   # expected: SBD-DC01.sbd.test
# On SBD-DC01 (the PDC emulator): serve time from its own clock as the reliable source
w32tm /config /reliable:yes /update
Restart-Service w32time
w32tm /query /source   # on SBD-DC01: Local CMOS Clock
# On SBD-DC02 and a member server:
w32tm /query /source   # expected: a domain controller
w32tm /query /status
w32tm /stripchart /computer:SBD-DC01.sbd.test /samples:3 /dataonly
```

> **WHAT SUCCESS LOOKS LIKE** SBD-DC01 reports itself as the reliable source (Local CMOS Clock), SBD-DC02 and the member server report a domain controller, host time synchronization is off for both domain controllers, and measured offsets sit inside the documented threshold.

Your record

Evidence to keep: Record the time design decisions (how DC01's clock is set and checked, and the drift threshold), the source and status output from both controllers and the member, the stripchart offsets, the hypervisor setting for both domain controller VMs, and the register rows that permit time: every member to the domain controllers.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Which system is the domain's authoritative time source in this design?

> **REMEMBER** SBD-DC01, the PDC emulator, is the declared source; how its clock is checked is written down. The hypervisor no longer overrides domain controller time; the hierarchy carries it everywhere. Offsets are measured and inside the threshold, not assumed.

Microsoft · Windows Time service

Microsoft · Windows Time service tools and settings

LESSON 06 / ABOUT 15 MIN

Find the fault and record the fix

WHY THIS MATTERS Troubleshoot one layer at a time: client settings, server reachability, authoritative data, replication, and finally the application that uses the name.

In this lesson you will

- Isolate a DNS failure one layer at a time.
- Distinguish a missing name (NXDOMAIN) from timeouts and service failures.
- Record the cause, the fix, and the rollback — not just a passing result.

Follow the build

1. For a failing lookup, record the exact name, record type, client, resolver, timestamp, and error. Compare the fully qualified name with any short-name search behavior.
2. Query both servers directly. If they disagree, inspect zone scope and replication. If both are correct but the client is not, inspect client DNS settings and cached data.
3. Distinguish a missing name (NXDOMAIN) from a timeout or service failure. Fix the owning configuration first, then use Clear-DnsClientCache on the test client if stale cache remains relevant.
4. Repeat the original test and the redundancy checks. Record what changed and the rollback method. Review aging/scavenging separately against DHCP, static records, and backup practices before enabling automatic deletion.

Check your result

```
Get-DnsClientServerAddress -AddressFamily IPv4
Get-DnsClientCache
Resolve-DnsName SBD-DC02.sbd.test -Server 10.0.10.10
Resolve-DnsName SBD-DC02.sbd.test -Server 10.0.10.11
```

WHAT SUCCESS LOOKS LIKE The original failure is reproducibly resolved, both resolvers agree, and the change record explains the cause rather than only showing a passing screenshot.

Your record

Evidence to keep: Complete the DNS checklist and retain the before/after lookup results with the configuration owner.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	

RECORD	YOUR OBSERVATION
Evidence file names	
Open items or findings	

Think it through

Both DNS servers return the correct address, but one client keeps using the old address. What should you inspect next?

> **REMEMBER** Troubleshoot client settings, reachability, authoritative data, replication, then the application. NXDOMAIN, timeouts, and service failures point at different owners. The change record explains the cause and the rollback.

Microsoft · Manage DNS resource records

Microsoft · Resolve-DnsName

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Map names to the network	☐ yes	
02 Configure forward and reverse zones	☐ yes	
03 Create and maintain service records	☐ yes	
04 Prove resolution and directory discovery	☐ yes	
05 Make SBD-DC01 the domain's time source	☐ yes	
06 Find the fault and record the fix	☐ yes	

Open items carried forward

Next in the learning path

That completes Configure DNS and time. Course 1.3, Harden the baseline, builds the organizational units and hardened security policies that every later system joins.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.